

03 — TEMPLATES, METAPROGRAMMING & CONCEPTS

Interviewers use templates to test whether you understand *when* code is generated, how overload/specialization resolution works, and modern constraint tools.

How to think about this. Split every template question into two clocks: *compile time* (the compiler stamps out a concrete function/class from the recipe) and *run time* (that stamped-out code executes). Most confusion — two-phase lookup, `if constexpr`, why errors fire late, code bloat — dissolves once you ask "which clock is this happening on?" Templates are a program that runs *on types* at compile time and emits the program that runs on *values*.

Take-home lesson. A template is not code — it's a *code generator*. Nothing exists until you instantiate it, errors in untaken branches don't fire, and every distinct instantiation is a separate compiled entity. Reason about templates in terms of "what gets stamped out, and when."

1. HOW TEMPLATES ACTUALLY WORK

- A template is a recipe. Nothing is compiled until instantiation with concrete args. Errors in un-instantiated branches don't fire — this is why `if constexpr` matters.
- Two-phase lookup: at definition, non-dependent names are bound; dependent names (those depending on a template param) are bound at instantiation. This is why you need `typename` for dependent types and `template` disambiguator for dependent member templates.

```
template <class T> void f(T t) {
    typename T::value_type v;           // 'typename' — dependent type
    t.template method<int>();           // 'template' disambiguator
}
```

- Each distinct instantiation is a separate entity → code bloat risk.
Mitigate with type erasure, `extern template`, or factoring non-dependent code into a non-template base.

War story. A "generic" container template with all logic inline compiled into dozens of near-identical copies (one per element type), ballooning the binary and thrashing the instruction cache. The fix: move every operation that *didn't* depend on `T` into a non-template base class, leaving the template as a thin typed wrapper. Binary shrank, build time dropped. The lesson: templates duplicate everything they touch — keep the type-dependent surface small.

2. DEDUCTION & SPECIALIZATION

Template argument deduction (function templates) strips top-level `const` and references from by-value params; deduces refs for `T&` / `T&&` . `auto` follows the same rules; `decltype(auto)` preserves them exactly.

- Class template argument deduction (CTAD, C++17): `std::pair p{1, 2.0};`
Deduction guides customize it.
- Specialization:
 - *Explicit (full) specialization* — one concrete set of args.
 - *Partial specialization* — for class templates only (not function templates; use overloading for functions).
 - Overload resolution + partial ordering picks the "most specialized".

```
template <class T> struct S { static constexpr int v = 0; };
template <class T> struct S<T*> { static constexpr int v = 1; }; //
partial spec
template <> struct S<int>      { static constexpr int v = 2; }; // full
spec
```

3. SFINAE → CONCEPTS (THE EVOLUTION)

SFINAE — "Substitution Failure Is Not An Error": an invalid substitution in a function template's signature removes it from the overload set instead of erroring. Old workhorse for constraints:

The catch: only the "immediate context" is soft. SFINAE forgives failures in the signature — return type, parameter types, template arguments — because those are what get substituted during overload resolution. An error deep in the function *body* is a hard error that halts compilation; it never triggers SFINAE. This fragility (constraint behavior depending on *where* the failure lands) is exactly why concepts and `if constexpr` supersede it: they put the check in a place with clean, predictable semantics.

```
template <class T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
void f(T);                                // only for integral T
```

Concepts (C++20) — named, composable constraints; far better errors and readability. Prefer them over `enable_if`.

```
template <class T>
concept Number = std::integral<T> || std::floating_point<T>;

template <Number T> T add(T a, T b) { return a + b; } // constrained
auto add2(Number auto a, Number auto b) { return a + b; } // abbreviated
```

- `requires` clause and `requires` expression (different things):

```
template <class T>
concept Addable = requires(T a, T b) {
    { a + b } -> std::convertible_to<T>; // compound requirement
};
```

- Concepts also drive subsumption — a more-constrained overload wins.

Stop and think. Why prefer a `concept` over the equivalent `enable_if` beyond "nicer syntax"?

4. CONSTEXPR / CONSTEVAL / COMPILE-TIME

- `constexpr` — *may* run at compile time if inputs are constant; else runtime.
- `constexpr` (C++20) — *immediate* function, must produce a constant.
- `constexpr` (C++20) — guarantees static init happens at compile time (kills SIOF for that variable) without forcing constness.
- `if constexpr` — compile-time branch; the *discarded* branch is not instantiated. Replaces most tag-dispatch and SFINAE-overload pairs.

```
template <class T> auto describe(T x) {  
    if constexpr (std::is_pointer_v<T>) return *x; // only compiled for  
    pointers  
    else  
        return x;  
}
```

C++20 pushed compile-time evaluation far further — even containers work:

```
constexpr int sum_upto(int n) { // MUST be evaluated at compile  
time  
    std::vector<int> v(n); // constexpr new/vector, C++20  
    std::iota(v.begin(), v.end(), 1);  
    return std::accumulate(v.begin(), v.end(), 0);  
}  
constexpr int s = sum_upto(100); // computed by the compiler →  
5050  
static_assert(s == 5050);
```

Take-home lesson. `if constexpr` collapsed a whole era of tag-dispatch and paired-SFINAE-overloads into one readable function. If you find yourself writing two overloads that differ only by a trait, reach for `if constexpr` first.

5. VARIADIC TEMPLATES & PARAMETER PACKS

```
template <class... Ts> void print(const Ts&... xs) {
    (std::cout << ... << xs);          // C++17 fold expression
}
```

- Fold forms: `(pack op ...)`, `(... op pack)`, `(init op ... op pack)`.
 - `sizeof...(Ts)` gives the count.
 - Pre-C++17 recursion (head/tail) is the classic pattern; folds replace most.
-

6. TYPE TRAITS & TAG DISPATCH (KNOW THE VOCABULARY)

- `<type_traits>`: `is_same`, `is_base_of`, `is_convertible`, `remove_cv`, `decay`, `conditional`, `enable_if`, `is_nothrow_move_constructible`, ...
`_v` suffix = value, `_t` suffix = type.
- Tag dispatch: select an overload via a trait-derived tag type (`std::random_access_iterator_tag` etc.). Largely superseded by `if constexpr`/concepts but still in STL internals.
- CRTP (Curiously Recurring Template Pattern): static polymorphism — `class D : Base<D>`. Zero-overhead alternative to virtual for compile-time known types; basis of `enable_shared_from_this`, expression templates.

```
template <class D> struct Base { void run(){ static_cast<D*>(this)->impl(); } };
struct D : Base<D> { void impl(){ /*...*/ } };
```

7. TYPE ERASURE

Hiding a concrete type behind a uniform interface while keeping value semantics. `std::function`, `std::any`, `std::shared_ptr`'s deleter all use it.

Pattern: an abstract inner "concept" + templated "model" holding the concrete object, owned via a pointer. Trades a virtual dispatch for flexibility.

The whole trick in ~20 lines — a value-semantic "any drawable":

```
class Drawable {
    struct Concept {                                // the erased interface
        virtual ~Concept() = default;
        virtual void draw() const = 0;
        virtual std::unique_ptr<Concept> clone() const = 0;
    };
    template <class T> struct Model : Concept {      // one per concrete
type
        T obj;
        explicit Model(T o) : obj(std::move(o)) {}
        void draw() const override { obj.draw(); } // calls T::draw — no
base class needed
        std::unique_ptr<Concept> clone() const override {
            return std::make_unique<Model>(*this);
        }
    };
    std::unique_ptr<Concept> self_;
public:
    template <class T>                             // accepts ANY type
with .draw()
        Drawable(T x) : self_(std::make_unique<Model<T>>(std::move(x))) {}
        Drawable(const Drawable& o) : self_(o.self_>clone()) {} // value
semantics
        Drawable(Drawable&&) noexcept = default;
        void draw() const { self_>draw(); }
};
// Circle and Square need NO common base, NO virtual — just a draw()
method.
std::vector<Drawable> shapes{Circle{}, Square{}}; // heterogeneous,
value-typed
for (const auto& s : shapes) s.draw();
```

Why this matters: the stored types share no inheritance relationship — the `virtual` lives entirely inside `Drawable`, not in `Circle` / `Square`. That's the difference from classic OOP polymorphism (File 06): the interface is *external* to the types, so you can retrofit it onto types you don't own (`int`, a third-party class) as long as they have the required operation.

SELF-CHECK LADDER (GRADE YOUR OWN MASTERY)

Level 1 — Recall

- What are the two phases of two-phase name lookup, and what happens in each?
- Which can be partially specialized: class templates, function templates, or both?
- What does `if constexpr` do to the *discarded* branch?

Level 2 — Apply

1. Explain two-phase lookup with a code sample that needs `typename` and one that needs the `template` disambiguator.
2. Convert an `enable_if` overload pair into `if constexpr`, then into concepts. Note what each version costs in readability and error quality.
3. Write a CRTP base that adds `operator!=` from a derived `operator==`.

Level 3 — Transfer

4. Why can't you partially specialize a function template? What do you use instead, and why does *that* work?
5. Implement `tuple`-like `get<N>` or a compile-time `for_each` over a parameter pack.
6. Sketch a minimal type-erased `Function<R(Args...)>` — identify exactly where the virtual dispatch hides.

Level 4 — Teach

- Explain to a colleague why a template error message is 200 lines long, and how concepts shorten it. If you can't point to *substitution* as the cause, revisit two-phase lookup.

CONNECTS TO

- File 02 — forwarding references (`T&&` deduced) and `std::forward` are template-deduction machinery.

- File 06 — CRTP here is the static-polymorphism counterpart to virtual dispatch there; both solve "call the right function," on different clocks.
- File 04 — tag dispatch and iterator categories are template metaprogramming the STL uses internally.
- File 08 — concepts, `if constexpr`, fold expressions, and CTAD are the modern-standard tools that replaced the old SFINAE patterns.